

書籍出版商管理系統 - JPA 一對多關聯教學

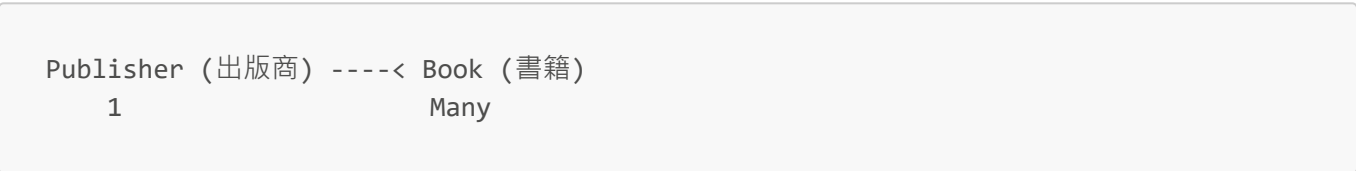
📖 學習目標

通過本教學，您將學會：

- 理解 JPA 中一對多(One-to-Many)與多對一(Many-to-One)的雙向關聯
- 掌握 `@OneToMany` 和 `@ManyToOne` 註解的使用方法
- 學習 CASCADE 級聯操作的應用
- 了解如何處理 JSON 序列化中的循環參照問題
- 建立完整的書籍出版商管理 REST API

🏠 系統架構概覽

本系統採用一對多關聯設計：



- 一個出版商可以出版多本書籍
- 每本書籍只能屬於一個出版商

📁 資料模型設計

資料表結構

publishers 表

欄位名	資料型別	說明
publisher_id	VARCHAR(10)	出版商編號（主鍵）
publisher_name	VARCHAR(100)	出版商名稱
address	VARCHAR(200)	地址
city	VARCHAR(50)	城市
country	VARCHAR(50)	國家
phone	VARCHAR(20)	電話
email	VARCHAR(100)	電子郵件
website	VARCHAR(100)	官方網站

books 表

欄位名	資料型別	說明
-----	------	----

欄位名	資料型別	說明
book_id	INT	書籍編號 (主鍵 · 自動遞增)
title	VARCHAR(200)	書名
author	VARCHAR(100)	作者
isbn	VARCHAR(20)	ISBN
price	DECIMAL(10,2)	價格
publish_date	DATE	出版日期
category	VARCHAR(50)	分類
publisher_id	VARCHAR(10)	出版商編號 (外鍵)

Entity 類別實作

1. Publisher.java - 出版商實體 (一端)

```
package model;

import java.io.Serializable;
import javax.persistence.*;
import java.util.List;
import java.util.ArrayList;

/**
 * 出版商實體類別
 */
@Entity
@Table(name="publishers")
@NamedQuery(name="Publisher.findAll", query="SELECT p FROM Publisher p")
public class Publisher implements Serializable {
    private static final long serialVersionUID = 1L;

    @Id
    @Column(name="publisher_id")
    private String publisherId;

    @Column(name="publisher_name")
    private String publisherName;

    private String address;
    private String city;
    private String country;
    private String phone;
    private String email;
    private String website;

    // 一對多雙向關聯 - 一個出版商可以有多本書
    @OneToMany(mappedBy="publisher", cascade=CascadeType.ALL,
```

```
fetch=FetchType.LAZY)
    private List<Book> books = new ArrayList<>();

    public Publisher() {
    }

    public Publisher(String publisherId, String publisherName, String address,
        String city, String country, String phone, String email,
String website) {
        this.publisherId = publisherId;
        this.publisherName = publisherName;
        this.address = address;
        this.city = city;
        this.country = country;
        this.phone = phone;
        this.email = email;
        this.website = website;
    }

    // Getter 和 Setter 方法
    public String getPublisherId() {
        return this.publisherId;
    }

    public void setPublisherId(String publisherId) {
        this.publisherId = publisherId;
    }

    public String getPublisherName() {
        return this.publisherName;
    }

    public void setPublisherName(String publisherName) {
        this.publisherName = publisherName;
    }

    public String getAddress() {
        return this.address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public String getCity() {
        return this.city;
    }

    public void setCity(String city) {
        this.city = city;
    }

    public String getCountry() {
        return this.country;
    }
```

```
}

public void setCountry(String country) {
    this.country = country;
}

public String getPhone() {
    return this.phone;
}

public void setPhone(String phone) {
    this.phone = phone;
}

public String getEmail() {
    return this.email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getWebsite() {
    return this.website;
}

public void setWebsite(String website) {
    this.website = website;
}

public List<Book> getBooks() {
    return this.books;
}

public void setBooks(List<Book> books) {
    this.books = books;
}

// 便利方法：新增書籍
public Book addBook(Book book) {
    getBooks().add(book);
    book.setPublisher(this);
    return book;
}

// 便利方法：移除書籍
public Book removeBook(Book book) {
    getBooks().remove(book);
    book.setPublisher(null);
    return book;
}

@Override
public String toString() {
```

```
        return "Publisher [publisherId=" + publisherId + ", publisherName=" +  
publisherName  
        + ", city=" + city + ", country=" + country + ", phone=" + phone +  
        "];"  
    }  
}
```

2. Book.java - 書籍實體 (多端)

```
package model;  
  
import java.io.Serializable;  
import javax.persistence.*;  
import java.math.BigDecimal;  
import java.time.LocalDate;  
import com.fasterxml.jackson.annotation.JsonIgnore;  
  
/**  
 * 書籍實體類別  
 */  
@Entity  
@Table(name="books")  
@NamedQuery(name="Book.findAll", query="SELECT b FROM Book b")  
public class Book implements Serializable {  
    private static final long serialVersionUID = 1L;  
  
    @Id  
    @GeneratedValue(strategy=GenerationType.IDENTITY)  
    @Column(name="book_id")  
    private int bookId;  
  
    private String title;  
    private String author;  
    private String isbn;  
    private BigDecimal price;  
  
    @Column(name="publish_date")  
    private LocalDate publishDate;  
  
    private String category;  
  
    // 多對一雙向關聯 - 多本書屬於一個出版商  
    @ManyToOne(fetch=FetchType.LAZY)  
    @JoinColumn(name="publisher_id", referencedColumnName="publisher_id")  
    @JsonIgnore // 防止 JSON 序列化時產生循環參照  
    private Publisher publisher;  
  
    public Book() {  
    }  
  
    public Book(String title, String author, String isbn, BigDecimal price,  

```

```
        LocalDate publishDate, String category) {
    this.title = title;
    this.author = author;
    this.isbn = isbn;
    this.price = price;
    this.publishDate = publishDate;
    this.category = category;
}

// Getter 和 Setter 方法
public int getBookId() {
    return this.bookId;
}

public void setBookId(int bookId) {
    this.bookId = bookId;
}

public String getTitle() {
    return this.title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getAuthor() {
    return this.author;
}

public void setAuthor(String author) {
    this.author = author;
}

public String getIsbn() {
    return this.isbn;
}

public void setIsbn(String isbn) {
    this.isbn = isbn;
}

public BigDecimal getPrice() {
    return this.price;
}

public void setPrice(BigDecimal price) {
    this.price = price;
}

public LocalDate getPublishDate() {
    return this.publishDate;
}
```

```
public void setPublishDate(LocalDate publishDate) {
    this.publishDate = publishDate;
}

public String getCategory() {
    return this.category;
}

public void setCategory(String category) {
    this.category = category;
}

public Publisher getPublisher() {
    return this.publisher;
}

public void setPublisher(Publisher publisher) {
    this.publisher = publisher;
}

@Override
public String toString() {
    return "Book [bookId=" + bookId + ", title=" + title + ", author=" +
author
        + ", isbn=" + isbn + ", price=" + price + ", category=" + category
+ "]\n";
}
```

關鍵註解說明

@OneToMany 註解 (在 Publisher 中)

```
@OneToMany(mappedBy="publisher", cascade=CascadeType.ALL, fetch=FetchType.LAZY)
private List<Book> books = new ArrayList<>();
```

- `mappedBy="publisher"`：指定關聯的擁有方是 Book 實體中的 `publisher` 屬性
- `cascade=CascadeType.ALL`：所有操作都會級聯到子實體
- `fetch=FetchType.LAZY`：延遲載入，提升效能

@ManyToOne 註解 (在 Book 中)

```
@ManyToOne(fetch=FetchType.LAZY)
@JoinColumn(name="publisher_id", referencedColumnName="publisher_id")
@JsonIgnore
private Publisher publisher;
```

- `@JoinColumn`：指定外鍵欄位名稱和參照欄位
- `@JsonIgnore`：防止 JSON 序列化時產生循環參照問題

Cascade 級聯類型

類型	說明
<code>CascadeType.PERSIST</code>	持久化時級聯
<code>CascadeType.MERGE</code>	更新時級聯
<code>CascadeType.REMOVE</code>	刪除時級聯
<code>CascadeType.REFRESH</code>	重新整理時級聯
<code>CascadeType.ALL</code>	所有操作都級聯

🌐 REST API 實作

PublisherService.java

```
import javax.ws.rs.*;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;
import model.*;
import java.util.*;

@Path("/publishers")
public class PublisherService {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public Response getAllPublishers() {
        PublisherDAO dao = new PublisherDAO();
        List<Publisher> publishers = dao.getAll();
        if(publishers.size() > 0) {
            return Response
                .status(Response.Status.OK)
                .entity(publishers)
                .build();
        } else {
            return Response
                .status(Response.Status.NO_CONTENT)
                .build();
        }
    }

    @GET
    @Path("/{id}")
    @Produces(MediaType.APPLICATION_JSON)
    public Response getPublisherById(@PathParam("id") String publisherId) {
        PublisherDAO dao = new PublisherDAO();
        Publisher publisher = dao.findById(publisherId);
    }
}
```



```
        if(publisher != null) {
            return Response
                .status(Response.Status.OK)
                .entity(publisher)
                .build();
        } else {
            return Response
                .status(Response.Status.NOT_FOUND)
                .build();
        }
    }

    @POST
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response addPublisher(Publisher publisher) {
        PublisherDAO dao = new PublisherDAO();
        boolean success = dao.addPublisher(publisher);
        if(success) {
            return Response
                .status(Response.Status.CREATED)
                .entity(publisher)
                .build();
        } else {
            return Response
                .status(Response.Status.BAD_REQUEST)
                .entity("新增出版商失敗")
                .build();
        }
    }

    @POST
    @Path("/with-books")
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response addPublisherWithBooks(Publisher publisher) {
        // 處理出版商和書籍的批次新增
        List<Book> books = publisher.getBooks();
        // 清除 JSON 中可能的循環參照
        books.forEach(book -> book.setPublisher(null));

        System.out.println("出版商資料: " + publisher.toString());
        System.out.println("書籍資料: " + books.toString());

        PublisherDAO dao = new PublisherDAO();
        boolean success = dao.addPublisherWithBooks(publisher);

        if(success) {
            return Response
                .status(Response.Status.CREATED)
                .entity("出版商和書籍新增成功")
                .build();
        } else {
            return Response
```

```
        .status(Response.Status.BAD_REQUEST)
        .entity("新增失敗")
        .build();
    }
}

@GET
@Path("/{id}/books")
@Produces(MediaType.APPLICATION_JSON)
public Response getBooksByPublisher(@PathParam("id") String publisherId) {
    PublisherDAO dao = new PublisherDAO();
    List<Book> books = dao.getBooksByPublisher(publisherId);
    if(books.size() > 0) {
        return Response
            .status(Response.Status.OK)
            .entity(books)
            .build();
    } else {
        return Response
            .status(Response.Status.NO_CONTENT)
            .build();
    }
}
```

DAO 層實作

PublisherDAO.java

```
package model;

import javax.persistence.*;
import java.util.*;

public class PublisherDAO {

    public EntityManager createConnection() {
        try {
            EntityManagerFactory factory =
                Persistence.createEntityManagerFactory("bookstore");
            EntityManager mgr = factory.createEntityManager();
            return mgr;
        } catch (Exception ex) {
            System.out.println("連線錯誤: " + ex.getMessage());
        }
        return null;
    }

    public List<Publisher> getAll() {
        EntityManager mgr = createConnection();
        TypedQuery<Publisher> query = mgr.createNamedQuery("Publisher.findAll",
```

```
Publisher.class);
    return query.getResultList();
}

public Publisher findById(String publisherId) {
    EntityManager mgr = createConnection();
    Publisher publisher = mgr.find(Publisher.class, publisherId);
    return publisher;
}

public boolean addPublisher(Publisher publisher) {
    EntityManager mgr = createConnection();
    mgr.getTransaction().begin();
    try {
        mgr.persist(publisher);
        mgr.getTransaction().commit();
        return true;
    } catch (Exception ex) {
        System.out.println("新增出版商錯誤: " + ex.getMessage());
        mgr.getTransaction().rollback();
        return false;
    }
}

public boolean addPublisherWithBooks(Publisher publisher) {
    EntityManager mgr = createConnection();
    mgr.getTransaction().begin();
    try {
        // 設定書籍與出版商的關聯
        List<Book> books = publisher.getBooks();
        for (Book book : books) {
            book.setPublisher(publisher);
        }

        // 由於設定了 CASCADE，只需要 persist 出版商
        mgr.persist(publisher);
        mgr.getTransaction().commit();
        return true;
    } catch (Exception ex) {
        System.out.println("新增出版商和書籍錯誤: " + ex.getMessage());
        mgr.getTransaction().rollback();
        return false;
    }
}

public List<Book> getBooksByPublisher(String publisherId) {
    String jpql = "SELECT b FROM Book b WHERE b.publisher.publisherId = :publisherId";
    EntityManager mgr = createConnection();
    TypedQuery<Book> query = mgr.createQuery(jpql, Book.class);
    query.setParameter("publisherId", publisherId);
    return query.getResultList();
}
```

```
public boolean deletePublisher(String publisherId) {
    EntityManager mgr = createConnection();
    Publisher publisher = mgr.find(Publisher.class, publisherId);
    if(publisher == null) return false;

    mgr.getTransaction().begin();
    try {
        // 由於設定了 CASCADE，刪除出版商會自動刪除相關書籍
        mgr.remove(publisher);
        mgr.getTransaction().commit();
        return true;
    } catch(Exception ex) {
        System.out.println("刪除出版商錯誤：" + ex.getMessage());
        mgr.getTransaction().rollback();
        return false;
    }
}
```

API 測試範例

1. 取得所有出版商

```
GET /publishers
Accept: application/json
```

2. 取得特定出版商

```
GET /publishers/PUB001
Accept: application/json
```

3. 新增出版商

```
POST /publishers
Content-Type: application/json

{
    "publisherId": "PUB001",
    "publisherName": "天下文化",
    "address": "台北市信義區松高路11號",
    "city": "台北",
    "country": "台灣",
    "phone": "02-2662-0012",
    "email": "info@cwgv.com.tw",
    "website": "https://www.cwgv.com.tw"
}
```

4. 同時新增出版商和書籍

POST /publishers/with-books

Content-Type: application/json

```
{
  "publisherId": "PUB002",
  "publisherName": "商周出版",
  "address": "台北市中山區民生東路二段141號",
  "city": "台北",
  "country": "台灣",
  "phone": "02-2500-7008",
  "email": "bwp@businessweekly.com.tw",
  "website": "https://www.businessweekly.com.tw",
  "books": [
    {
      "title": "Java 程式設計入門",
      "author": "張三",
      "isbn": "978-986-272-123-4",
      "price": 450.00,
      "publishDate": "2024-01-15",
      "category": "程式設計"
    },
    {
      "title": "Spring Boot 實戰指南",
      "author": "李四",
      "isbn": "978-986-272-567-8",
      "price": 520.00,
      "publishDate": "2024-03-20",
      "category": "程式設計"
    }
  ]
}
```

5. 取得特定出版商的所有書籍

GET /publishers/PUB002/books

Accept: application/json

⚠ 常見問題與解決方案

1. JSON 循環參照問題

問題： Publisher 包含 Books，而每個 Book 又包含 Publisher，造成無限循環。

解決方案：

- 在 Book 實體的 publisher 屬性上加上 @JsonIgnore
- 或使用 @JsonManagedReference 和 @JsonBackReference

2. 延遲載入問題

問題：訪問關聯資料時出現 LazyInitializationException。

解決方案：

- 在需要時使用 JOIN FETCH 查詢
- 或在 DAO 方法中確保 EntityManager 仍在作用中

3. 級聯刪除風險

問題：刪除出版商時意外刪除所有相關書籍。

解決方案：

- 謹慎使用 CascadeType.REMOVE
- 可考慮先檢查是否有關聯資料，或使用軟刪除

學習重點整理

1. 關聯設計原則

- 一端 (Publisher) 使用 @OneToMany，通常設定 mappedBy
- 多端 (Book) 使用 @ManyToOne，通常是關聯的擁有方
- 善用 cascade 和 fetch 屬性優化效能

2. 資料操作技巧

- 新增關聯資料時要正確設定雙向關聯
- 利用便利方法 (addBook、removeBook) 維護資料一致性
- 適當使用 JPQL 查詢關聯資料

3. JSON 處理要點

- 防止循環參照問題
- 處理接收到的 JSON 資料時清除可能的關聯設定
- 考慮使用 DTO 模式分離展示層和資料層

進階練習建議

1. 書籍搜尋功能：實作依書名、作者、ISBN 搜尋書籍的 API
2. 分頁查詢：為出版商和書籍列表加上分頁功能
3. 統計資訊：統計每個出版商的書籍數量和總價值
4. 批次操作：實作批次更新書籍價格的功能
5. 資料驗證：加入 Bean Validation 確保資料正確性
6. 軟刪除：改用軟刪除避免資料遺失
7. 審計功能：記錄資料的建立和修改時間

這個書籍出版商管理系統展示了 JPA 一對多關聯的完整應用，是學習企業級 Java 開發不可或缺的重要範例！